

Assignment 1(b)

Title: Parallel Depth First Search based on existing algorithms using OpenMP

Name: Sakshi Kaspate

Roll no: 14208

Div: B

Batch: B1

```
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>

using namespace std;

const int MAX = 100000;

vector<int> graph[MAX];
bool visited[MAX];

/* Parallel DFS */
void dfs(int start_node)
{
    stack<int> s;
    s.push(start_node);

    while (!s.empty())
    {
        int curr_node;

        /* Critical section for stack access */
        #pragma omp critical
        {
            if (!s.empty()) {
                curr_node = s.top();
                s.pop();
            }
            else
                curr_node = -1;
        }

        if (curr_node == -1)
            continue;

        if (!visited[curr_node])
        {
```

```

        visited[curr_node] = true;
        cout << curr_node << " ";

        /* Explore neighbors in parallel */
        #pragma omp parallel for
        for (int i = 0; i < graph[curr_node].size(); i++)
        {
            int adj_node = graph[curr_node][i];

            if (!visited[adj_node])
            {
                #pragma omp critical
                s.push(adj_node);
            }
        }
    }
}

int main()
{
    int n, m, start_node;

    cout << "Enter number of nodes, edges, start node: ";
    cin >> n >> m >> start_node;

    cout << "Enter edge pairs:\n";
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    /* Initialize visited array in parallel */
    #pragma omp parallel for
    for (int i = 0; i < n; i++)
        visited[i] = false;

    cout << "DFS Traversal:\n";
    dfs(start_node);

    return 0;
}

```

OUTPUT:

```
datanalytics1@datanalytics1-OptiPlex-7050:~$ ./parallel_dfs
Enter No of Nodes, Edges and Start Node: 6 5 0
Enter Pair of Edges:
0 1
0 2
1 3
3 4
3 5

Parallel DFS Traversal:
0 1 3 5 4 2 datanalytics1@datanalytics1-OptiPlex-7050:~$
```